

CM1 - Introduction à la Programmation C

Ammar Mian

ammam.mian@univ-smb.fr

Bureau A136



Plan du cours

Informations pratiques

Organisation du cours

- 8 Cours Magistraux (CMs)
- 7 Travaux Dirigés (TDs)
- 2 Travaux Pratiques (TPs)
- 1 Projet (fin de semestre : CM/TD + travail personnel)

Évaluation

- Examen final
- Examen de TP

En cas de difficulté

Me contacter rapidement !

Bureau A136 ou ammar.mian@univ-smb.fr

Questionnaire étudiant

Merci de remplir ce questionnaire
pour que je puisse mieux connaître
vos attentes et votre niveau



Objectifs du CM1

À l'issue de ce cours, vous serez capables de :

- **Comprendre** ce qu'est la programmation informatique
- **Découvrir** le langage C et son histoire
- **Maîtriser** les éléments de base : variables, types, opérateurs
- **Utiliser** les structures de contrôle (if, for, while)
- **Écrire** vos premières fonctions
- **Compiler** et exécuter votre premier programme

"La programmation s'apprend en pratiquant !"

1. Qu'est-ce que la programmation ?

2. Le langage C

3. Éléments de base du C

4. Structures de contrôle

5. Compilation et exécution

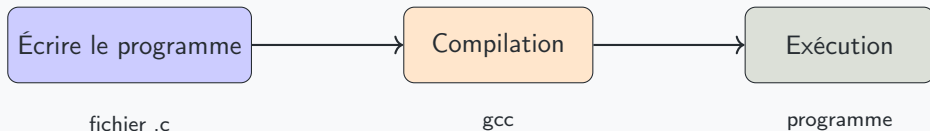
6. Les fonctions

7. Exemple : Le jeu de Nim

Définition et pipeline

Programmation

Activité consistant à créer un **fichier texte** contenant des **instructions** que l'ordinateur doit suivre pour accomplir une tâche.



Algorithmique et structures de données

Algorithmique

- Science qui étudie les **algorithmes**
- Algorithme = **méthode** pour résoudre un problème
- Exemple : recette de cuisine

Structures de données

- **Organisation** pour stocker l'information
- Exemples : tableaux, listes, piles, files
- Optimisent les opérations

Algorithmique + Structures de données = Programme

Exemple motivant : Les fractales

Ensemble de Mandelbrot

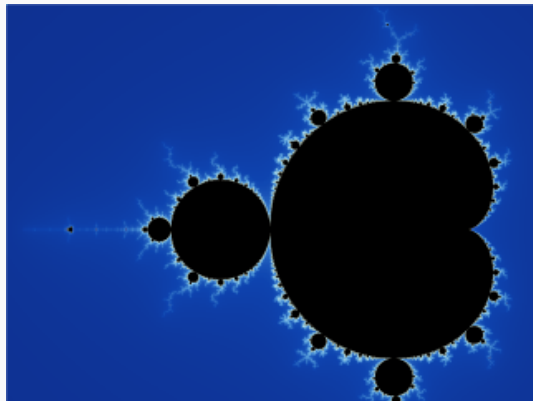
Suite complexe définie par :

$$z_0 = 0$$

$$z_{n+1} = z_n^2 + c$$

L'ensemble = points c pour lesquels la suite reste bornée

Visualisation impossible sans programmation !



Source: Wikipedia

Propriété d'auto-similarité : même structure à différentes échelles

Applications de la programmation

Sciences et ingénierie

- Simulations physiques (climat, mécanique)
- Chimie computationnelle
- Astronomie (trajectoires)
- Traitement du signal

Technologies

- Intelligence artificielle
- Robotique
- Systèmes embarqués

Vie quotidienne

- Applications mobiles
- Jeux vidéo
- Sites web
- Réseaux sociaux

Analyse de données

- Big Data
- Statistiques
- Visualisation
- Machine Learning

Transition

Comment écrire ces programmes ?



Il faut un **langage de programmation**



Nous utiliserons le C

1. Qu'est-ce que la programmation ?

2. Le langage C

3. Éléments de base du C

4. Structures de contrôle

5. Compilation et exécution

6. Les fonctions

7. Exemple : Le jeu de Nim

Histoire et caractéristiques

Histoire

- Créé en **1972** par Dennis Ritchie
- Bell Labs (AT&T)
- Conçu pour écrire UNIX

Caractéristiques

- Langage de “**bas niveau**”
- Contrôle précis de la machine
- **Très rapide et efficace**
- Portable



Dennis Ritchie (gauche) et Ken Thompson (droite),
1972

Pourquoi apprendre le C ?

1. Comprendre comment fonctionne un ordinateur

- Accès direct à la mémoire
- Gestion explicite des ressources

2. Base pour apprendre d'autres langages

- C++, Java, C#, Python utilisent des concepts similaires

3. Performance

- Programmes très rapides
- Idéal pour calcul haute performance

4. Très utilisé en industrie

- Systèmes d'exploitation (Linux, Windows)
- Logiciels embarqués (automobile, aéronautique)
- Drivers, bibliothèques système

Hello World

Votre premier programme en C !

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello world!\n");
5     return 0;
6 }
```

C'est tout ce qu'il faut pour écrire son premier programme !

Anatomie du programme

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello world!\n");
5     return 0;
6 }
```

- Directive de préprocesseur
- Inclut la bibliothèque standard d'entrée/sortie
- Permet d'utiliser printf, scanf, etc.

```
1 #include <stdio.h>
2
3 int main() { ... }
```

- Point d'entrée du programme
- int = type de retour (entier)
- main = nom spécial reconnu par le compilateur
- Accolades { } délimitent le corps

Anatomie du programme (suite)

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello world!\n");
5     return 0;
6 }
```

- Fonction d'affichage
- Chaîne de caractères entre guillemets
- `\n` = retour à la ligne
- Point-virgule **obligatoire** à la fin

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello world!\n");
5     return 0;
6 }
```

- Retourne 0 au système d'exploitation¹³

1. Qu'est-ce que la programmation ?

2. Le langage C

3. Éléments de base du C

4. Structures de contrôle

5. Compilation et exécution

6. Les fonctions

7. Exemple : Le jeu de Nim

Types de base

Type	Usage	Exemple	Spécificateur
int	Entiers	42, -10	%d
float	Décimaux	3.14	%f
double	Haute précision	3.14159	%lf
char	Caractères	'A', 'z'	%c

Remarques

- int : généralement 4 octets (32 bits)
- float : 4 octets, précision 7 chiffres
- double : 8 octets, précision 15 chiffres
- char : 1 octet, pour les caractères ASCII

Déclaration et initialisation

```
1 // Déclaration seule
2 int age;
3 age = 20; // Affectation
4
5 // Déclaration avec initialisation
6 int nombre = 10;
7 float pi = 3.14;
8 double e = 2.71828;
9 char lettre = 'A';
10
11 // Plusieurs variables du meme type
12 int x = 5, y = 10, z = 15;
```

Attention

Une variable non initialisée contient une **valeur aléatoire** !

Printf avec variables

```
1 #include <stdio.h>
2
3 int main() {
4     int age = 20;
5     float taille = 1.75;
6     char initiale = 'A';
7
8     printf("J'ai %d ans\n", age);
9     printf("Je mesure %.2f m\n", taille);
10    printf("Mon initiale est %c\n", initiale);
11
12    return 0;
13 }
```

Output :

J'ai 20 ans

Je mesure 1.75 m

Mon initiale est A

- %.2f = 2 chiffres après la virgule

Opérations arithmétiques

```
1 int a = 10, b = 3;
2
3 int somme = a + b;           // 13
4 int difference = a - b;     // 7
5 int produit = a * b;        // 30
6 int quotient = a / b;       // 3 (division entiere!)
7 int reste = a % b;          // 1 (modulo)
8
9 // Operations composees
10 int x = 5;
11 x = x + 3;                  // 8
12 x += 3;                     // Equivalent : 11
13 x *= 2;                     // 22
```

Division entière

La division de deux entiers donne un entier !

$10 / 3 = 3$ (pas 3.333...)

Attention : division entière !

Division entière

```
1 int a = 10, b = 3;  
2 int resultat = a / b;  
3 // resultat = 3
```

La partie décimale est **tronquée**

Division flottante

```
1 int a = 10, b = 3;  
2 float resultat = (float)a / b;  
3 // resultat = 3.333...
```

Le **cast** (float) est nécessaire !

Règle de la division

Règle importante

- Si les **deux opérandes** sont entiers → **division entière**
- Si **au moins un** est flottant → **division flottante**

Attention

$10 / 3 = 3$ (et non 3.333...)

Pour obtenir un résultat flottant, il faut caster :

$(\text{float})10 / 3 = 3.333\dots$

Incrémentation et décrémentation

```
1 int i = 5;
2
3 i = i + 1;           // i devient 6
4 i += 1;              // Equivalent, i devient 7
5 i++;                // Post-incrementation, i devient 8
6 ++i;                // Pre-incrementation, i devient 9
7
8 // Pareil pour la decrementation
9 i--;                // Post-decrementation, i devient 8
10 --i;               // Pre-decrementation, i devient 7
11 i -= 2;            // i devient 5
```

Différence pré/post

- ++i : incrémente **puis** utilise la valeur
- i++ : utilise la valeur **puis** incrémente

Opérateurs de comparaison

```
1 int a = 10, b = 5;
2
3 int test1 = (a == b);    // Egalite : 0 (false)
4 int test2 = (a != b);    // Difference : 1 (true)
5 int test3 = (a > b);     // Supérieur : 1 (true)
6 int test4 = (a < b);     // Inférieur : 0 (false)
7 int test5 = (a >= b);    // Supérieur ou égal : 1
8 int test6 = (a <= b);    // Inférieur ou égal : 0
```

Convention en C

- 0 = false (faux)
- Toute valeur non-zéro = true (vrai)

Note

Attention : == (comparaison) $\neq$ = (affectation)
21

Opérateurs logiques

```
1 int a = 1, b = 0;
2
3 // ET logique (AND)
4 int test1 = a && b;           // 0 (false)
5 int test2 = a && 1;           // 1 (true)
6
7 // OU logique (OR)
8 int test3 = a || b;           // 1 (true)
9 int test4 = b || 0;           // 0 (false)
10
11 // NON logique (NOT)
12 int test5 = !a;               // 0 (false)
13 int test6 = !b;               // 1 (true)
```

Table de vérité ET

```
true && true = true
true && false = false
false && false = false
```

Commentaires

```
1 // Commentaire sur une seule ligne
2
3 /*
4  * Commentaire sur
5  * plusieurs lignes
6  */
7
8 int age = 20; // Commentaire en fin de ligne
9
10 /* On peut aussi commenter
11    du code pour le désactiver
12    temporairement */
13 // int x = 5;
```

Bonnes pratiques

- Commenter le **pourquoi**, pas le **quoi**
- Expliquer la logique, pas répéter le code
- Mettre à jour les commentaires avec le code

Règles de nommage

Règles syntaxiques

- Commence par une **lettre** ou `_`
- Peut contenir lettres, chiffres, `_`
- **Sensible à la casse** : `age` $\neq$ `Age`
- Pas de mots-clés : `int`, `float`, `return`, etc.

Bonnes pratiques

- Noms **descriptifs** : `nombre_etudiants` plutôt que `n`
- Style **snake_case** : `calcul_moyenne`
- Constantes en MAJUSCULES : `MAX_SIZE`

```

1 int nombre_etudiants = 50;           // 24 Bon
2 int x123abc;                        // Legal mais pas clair
    
```

Exercice 1

À vous de jouer ! (5 minutes)

Écrire un programme qui :

1. Déclare deux variables entières a et b
2. Les initialise avec les valeurs 15 et 4
3. Calcule et affiche :
 - Leur somme
 - Leur différence
 - Leur produit
 - Leur quotient (division entière)
 - Le reste de la division

Output attendu :

15 + 4 = 19

15 - 4 = 11

15 * 4 = 60

15 / 4 = 3

1. Qu'est-ce que la programmation ?

2. Le langage C

3. Éléments de base du C

4. Structures de contrôle

5. Compilation et exécution

6. Les fonctions

7. Exemple : Le jeu de Nim

Instruction conditionnelle : if

```
1 int age = 18;  
2  
3 if (age >= 18) {  
4     printf("Majeur\n");  
5 }
```

- Exécute le bloc si la condition est vraie
- Si la condition est fausse, le bloc est ignoré

Syntaxe

```
if (condition) { instructions }
```


Instruction if-else

```
1 int age = 15;
2
3 if (age >= 18) {
4     printf("Majeur\n");
5 } else {
6     printf("Mineur\n");
7 }
```

- Si condition vraie → exécute le premier bloc
- Si condition fausse → exécute le bloc else

Syntaxe

```
if (condition) { instructions } else { instructions }
```

if-else if-else : Exemple

```
1 int note = 15;
2
3 if (note >= 16) {
4     printf("Excellent!\n");
5 } else if (note >= 14) {
6     printf("Tres bien!\n");
7 } else if (note >= 12) {
8     printf("Bien\n");
9 } else if (note >= 10) {
10    printf("Passable\n");
11 } else {
12    printf("Insuffisant\n");
13 }
```

Pour note = 15 : affiche "Tres bien!"

if-else if-else : Fonctionnement

- On peut enchaîner plusieurs `else if`
- Le **premier bloc** dont la condition est vraie est exécuté
- Les **autres blocs sont ignorés** (même si leur condition est vraie)
- Le bloc `else final` est exécuté si **aucune** condition n'est vraie
- Le `else final` est optionnel

Utilisation

Idéal pour tester plusieurs conditions mutuellement exclusives

Boucle for - Syntaxe

Syntaxe générale

```
1 for (initialisation; condition; increment) {  
2     // Instructions à répéter  
3 }
```

Exemple

```
1 for (int i = 0; i < 5; i++) {  
2     printf("i vaut %d\n", i);  
3 }
```

Output :

```
i vaut 0  
i vaut 1  
i vaut 2  
i vaut 3  
i vaut 4
```

1. `int i = 0` : Initialisation

- Exécutée une seule fois au début
- Crée et initialise la variable

2. `i < 5` : Condition

- Testée avant chaque itération
- Si vraie : exécute le corps
- Si fausse : sort de la boucle

3. `i++` : Incrément

- Exécuté après chaque itération
- Modifie la variable de contrôle

Boucle while

```
1 int i = 0;
2 while (i < 5) {
3     printf("i vaut %d\n", i);
4     i++;
5 }
```

- Répète tant que la condition est **vraie**
- La condition est testée **avant** chaque itération
- Plus flexible que for pour conditions complexes

Attention

Ne pas oublier de modifier la condition dans le corps !
Sinon → boucle infinie

```
1 // Boucle infinie !
2 while (i < 5) {
3     printf("i vaut %d\n", i);
4     // Oubli du i++
```

Boucle do-while

```
1 int i = 0;
2 do {
3     printf("i vaut %d\n", i);
4     i++;
5 } while (i < 5);
```

Différence avec while

- La condition est testée **après** chaque itération
- Le corps est exécuté **au moins une fois**

Important

Même si la condition est fausse dès le départ,
le corps est exécuté au moins une fois !

do-while : Cas d'usage

Validation d'entrée utilisateur

Utile pour redemander jusqu'à obtenir une valeur valide :

```
1 int choix;  
2 do {  
3     printf("Entrez 1, 2 ou 3 : ");  
4     scanf("%d", &choix);  
5 } while (choix < 1 || choix > 3);
```

- L'utilisateur est interrogé au moins une fois
- Si la valeur est invalide, on redemande
- On sort de la boucle quand l'entrée est valide

break et continue

break : sort de la boucle

```
1 for (int i = 0; i < 10; i++) {  
2     if (i == 5) {  
3         break;  
4     }  
5     printf("%d ", i);  
6 }  
7 // Output: 0 1 2 3 4
```

continue : passe à l'itération suivante

```
1 for (int i = 0; i < 10; i++) {  
2     if (i % 2 == 0) {  
3         continue;  
4     }  
5     printf("%d ", i);  
6 }  
7 // Output: 1 3 5 7 9
```

Utilisation

- **break** : sortir prématurément d'une boucle
- **continue** : sauter le reste de l'itération actuelle

Exercice 2

À vous de jouer ! (5 minutes)

Écrire un programme qui affiche la table de multiplication de 7
(de 1 à 10)

Output attendu :

7 x 1 = 7

7 x 2 = 14

7 x 3 = 21

7 x 4 = 28

7 x 5 = 35

7 x 6 = 42

7 x 7 = 49

7 x 8 = 56

7 x 9 = 63

7 x 10 = 70

1. Qu'est-ce que la programmation ?

2. Le langage C

3. Éléments de base du C

4. Structures de contrôle

5. Compilation et exécution

6. Les fonctions

7. Exemple : Le jeu de Nim

Compilation avec gcc

Étapes

1. Écrire le programme

```
1 // fichier: hello.c
2 #include <stdio.h>
3
4 int main() {
5     printf("Hello world!\n");
6     return 0;
7 }
```

2. Compiler

```
1 gcc -o hello hello.c
```

- gcc : le compilateur
- -o hello : nom de l'exécutable
- hello.c : fichier source

Erreurs de compilation courantes

```
1 // Oubli du point-virgule
2 printf("Hello")          // ERREUR
3 // error: expected ';' before 'return'
4
5 // Oubli des guillemets
6 printf(Hello);           // ERREUR
7 // error: 'Hello' undeclared
8
9 // Mauvaise casse
10 Printf("Hello");        // ERREUR
11 // warning: implicit declaration of function 'Printf'
12
13 // Oubli de l'include
14 // printf sera "inconnu"
```

Conseil

Le compilateur indique la **ligne** et le **type** d'erreur.
Lire attentivement les messages !

Demo live + À vous de jouer

Démonstration en direct

Je vais compiler et exécuter un programme
sous vos yeux

Puis à vous !

Exercice pratique (5 minutes)

1. Créer un fichier `bonjour.c`
2. Écrire un programme qui affiche votre nom
3. Compiler avec `gcc`
4. Exécuter et vérifier le résultat

1. Qu'est-ce que la programmation ?

2. Le langage C

3. Éléments de base du C

4. Structures de contrôle

5. Compilation et exécution

6. Les fonctions

7. Exemple : Le jeu de Nim

Pourquoi des fonctions ?

1. Réutiliser du code

- Éviter de répéter les mêmes instructions

2. Organiser le programme

- Diviser en tâches logiques
- Faciliter la lecture et la maintenance

3. Abstraire la complexité

- Cacher les détails d'implémentation
- Interface claire : entrées → sortie

4. Tester plus facilement

- Chaque fonction peut être testée indépendamment

"Une fonction = une tâche précise"

Anatomie d'une fonction

```
1 int addition(int a, int b) {  
2     return a + b;  
3 }
```

- `int` : Type de retour
 - Type de la valeur renvoyée par la fonction
- `addition` : Nom de la fonction
 - Identifiant unique et descriptif
- `int a, int b` : Paramètres (entrées)
 - Données nécessaires pour la fonction
- `return a + b` : Valeur de retour
 - Résultat calculé par la fonction

Prototype et appel

```
1 // Prototype (declaration) en haut du fichier
2 int addition(int a, int b);
3
4 // Definition de la fonction
5 int addition(int a, int b) {
6     return a + b;
7 }
8
9 // Appel dans le main
10 int main() {
11     int somme = addition(5, 3);
12     printf("5 + 3 = %d\n", somme);
13
14     // Ou directement
15     printf("10 + 7 = %d\n", addition(10, 7));
16
17     return 0;
18 }
```

- **Prototype** : déclare l'existence de la fonction
- **Définition** : contient le code de la fonction
- **Appel** : exécute la fonction avec des valeurs

Fonction sans retour (void)

```
1 void afficher_message(void) {  
2     printf("Bonjour depuis la fonction!\n");  
3 }  
4  
5 int main() {  
6     afficher_message();    // Pas de return  
7     afficher_message();    // On peut l'appeler plusieurs fois  
8  
9     return 0;  
10 }
```

Output :

Bonjour depuis la fonction!

Bonjour depuis la fonction!

- void = pas de valeur de retour
- La fonction effectue une action (affichage, calcul, etc.)
- Pas de return valeur (juste return; optionnel)

Fonction sans paramètres

```
1 int obtenir_reponse(void) {  
2     return 42;  
3 }  
4  
5 int main() {  
6     int reponse = obtenir_reponse();  
7     printf("La reponse est %d\n", reponse);  
8  
9     return 0;  
10 }
```

- void entre parenthèses = aucun paramètre
- La fonction peut quand même retourner une valeur
- Utile pour des valeurs constantes ou des générateurs

Fonction avec plusieurs paramètres

```
1 float moyenne(float a, float b, float c) {
2     return (a + b + c) / 3.0;
3 }
4
5 float maximum(float x, float y, float z) {
6     float max = x;
7     if (y > max) max = y;
8     if (z > max) max = z;
9     return max;
10 }
11
12 int main() {
13     float moy = moyenne(12.5, 15.0, 14.5);
14     printf("Moyenne : %.2f\n", moy);
15
16     float max = maximum(10.5, 23.7, 18.2);
17     printf("Maximum : %.2f\n", max);
18
19     return 0;
20 }
```

Passage par valeur

```
1 void modifier(int x) {  
2     x = 100;           // Modifie la copie locale  
3     printf("Dans la fonction : x = %d\n", x);  
4 }  
5  
6 int main() {  
7     int a = 10;  
8     printf("Avant : a = %d\n", a);  
9  
10    modifier(a);  
11  
12    printf("Après : a = %d\n", a);    // a vaut toujours 10!  
13  
14    return 0;  
15 }
```

Output :

Avant : a = 10

Dans la fonction : x = 100

Après : a = 10

Exercice 3

À vous de jouer ! (5 minutes)

Écrire une fonction max qui :

- Prend deux entiers en paramètres
- Retourne le plus grand des deux

Tester dans le main :

```
1 int main() {  
2     printf("Max(10, 20) = %d\n", max(10, 20));  
3     printf("Max(50, 30) = %d\n", max(50, 30));  
4     printf("Max(-5, -10) = %d\n", max(-5, -10));  
5     return 0;  
6 }
```

Output attendu :

Max(10, 20) = 20

Max(50, 30) = 50

Max(-5, -10) = -5

Solution + Aperçu récursivité

Solution exercice 3

```
1 int max(int a, int b) {
2     if (a > b) {
3         return a;
4     } else {
5         return b;
6     }
7 }
8
9 // Version compacte
10 int max(int a, int b) {
11     return (a > b) ? a : b;
12 }
```

Aperçu : récursivité

```
1 int factorielle(int n) {
2     if (n <= 1) {
3         return 1;
4     }
5     return n * factorielle(n-1);
6 }
7
8 // Exemple: factorielle(5)
9 // = 5 * factorielle(4)
10 // = 5 * 4 * factorielle(3)
11 // = 5 * 4 * 3 * 2 * 1
12 // = 120
```

- Une fonction peut s'appeler elle-même
- On reviendra sur ce concept plus tard

1. Qu'est-ce que la programmation ?

2. Le langage C

3. Éléments de base du C

4. Structures de contrôle

5. Compilation et exécution

6. Les fonctions

7. Exemple : Le jeu de Nim

Le jeu de Nim - Règles

Règles

- 12 bâtonnets au départ
- 2 joueurs alternent
- Chacun peut retirer 1, 2 ou 3 bâtonnets
- Celui qui prend le **dernier** bâtonnet **gagne**

Exemple de partie :

- Départ : 12 bâtonnets
- J1 prend 2 $\rightarrow$ reste 10
- J2 prend 3 $\rightarrow$ reste 7
- J1 prend 1 $\rightarrow$ reste 6
- J2 prend 2 $\rightarrow$ reste 4



Le jeu de Nim physique

Structure du programme

```
1 #include <stdio.h>
2
3 int main() {
4     int batons = 12;
5     int joueur = 0;    // 0 ou 1
6     int pris;
7
8     printf("Bienvenue dans le jeu de NIM!\n\n");
9
10    while (batons > 0) {
11        afficher_batonnets(batons);
12        printf("Joueur %d : ", joueur + 1);
13        pris = prendre_batons(batons);
14        batons -= pris;
15        joueur = (joueur + 1) % 2;    // Alterne 0 et 1
16    }
17
18    printf("\nBravo ! Joueur %d a gagne!\n",
19          (joueur + 1) % 2);
20
21    return 0;
22 }
```

Fonction d'affichage

```
1 void afficher_batonnets(int nombre) {  
2     printf("Batons restants : %d\n", nombre);  
3  
4     // Afficher les batons visuellement  
5     for (int i = 0; i < nombre; i++) {  
6         printf("|");  
7     }  
8     printf("\n");  
9 }
```

Exemple d'affichage :

Batons restants : 7

|||||||

- Utilise une boucle for pour afficher les |
- printf sans \n reste sur la même ligne

Fonction de saisie avec validation

```
1 int prendre_batons(int nombre_restant) {
2     int pris;
3
4     do {
5         printf("Combien de batons (1-3) ? ");
6         scanf("%d", &pris);
7
8         // Verification de la validite
9         if (pris < 1 || pris > 3) {
10             printf("Erreur : entre 1 et 3 seulement!\n");
11         } else if (pris > nombre_restant) {
12             printf("Erreur : il reste seulement %d batons!\n",
13                 nombre_restant);
14         }
15
16     } while (pris < 1 || pris > 3 || pris > nombre_restant);
17
18     return pris;
19 }
```

- Boucle do-while pour redemander si invalide
- scanf pour lire l'entrée utilisateur

Démo en direct

Démonstration

Je vais exécuter le programme
et jouer quelques tours

Code complet disponible sur la page web du cours

Ce qu'on a appris aujourd'hui

- ✓ **Principes** de la programmation informatique
- ✓ Le **langage C** : histoire, caractéristiques
- ✓ **Types de base** : int, float, double, char
- ✓ **Variables** : déclaration, initialisation, affichage
- ✓ **Opérations** : arithmétiques, comparaison, logique
- ✓ **Structures de contrôle** : if, for, while
- ✓ **Fonctions** : définition, appel, paramètres, retour
- ✓ **Compilation** avec gcc
- ✓ **Exemple pratique** : jeu de Nim complet

Vous savez maintenant écrire vos premiers programmes en C !

Pour la suite + Questions

La suite du cours

- **CM2** : Représentation des données en mémoire
 - Codage binaire, entiers, flottants, caractères
- **TD1** : Exercices sur les bases
 - Variables, boucles, fonctions

Ressources

- Page web du cours : [URL à compléter]
- Email : `ammar.mian@univ-smb.fr`
- Bureau : A136
- Livre de référence : K&R "The C Programming Language"

Questions ?

Merci !

Bon courage pour les TD

À la semaine prochaine pour le CM2

`ammar.mian@univ-smb.fr`

Bureau A136